Stolpe,
Graal-LSP

Graal-LSP: A Zero-overhead Polyglot Language Server Implementation for the GraalVM

Graal-LSP: Eine Language Server Implementierung für die GraalVM ohne Beinträchtigung der Ausführungsgeschwindigkeit

by

Daniel Stolpe

A thesis submitted to the
Hasso Plattner Institute
at the University of Potsdam, Germany
in partial fulfillment of the requirements for the degree of

Master of Science in IT-Systems Engineering

Supervisors

Prof. Dr. Robert Hirschfeld
Dr. John Doe

Software Architecture Group
Hasso Plattner Institute
University of Potsdam, Germany

December 26, 2018

Abstract

The english abstract.

Zusammenfassung

Die Zusammenfassung auf deutsch.

Acknowledgments

I owe everything to my cat.

Contents

1. Introduction **1**
1.1. Contributions . . . 2
1.2. Outline . . . 3

2. Context **5**
2.1. Source Code Editors . . . 5
2.2. Language Server Protocol . . . 6
2.3. Terms and Concepts . . . 8
2.4. Language Implementation Frameworks . . . 9

3. Approach **13**
3.1. Completion of Globals and Locals . . . 13
3.2. Obtaining and Using Run-time Information . . . 14
3.3. Completion of Properties . . . 17
3.4. Transferring the Approach to Other Requests . . . 19

4. Implementation **23**
4.1. Prerequisites to Answer LSP Requests with Truffle . . . 23
4.1.1. Project Structure and Dependencies . . . 23
4.1.2. Threads and Context Activation . . . 24
4.1.3. Language Server Initialization . . . 25
4.1.4. Parsing Source Code . . . 25
4.1.5. Mapping Requested Source Position to AST Node . . . 26
4.1.6. Obtaining Run-time Information . . . 26
4.2. Implementing LSP Request Handlers . . . 28
4.2.1. Completion of Globals and Locals . . . 28
4.2.2. Completion of Properties . . . 30
4.2.3. Signature Help . . . 32
4.2.4. Hover Information . . . 32
4.2.5. Document Symbols . . . 33
4.2.6. Workspace Symbols . . . 33
4.2.7. Document Highlight . . . 33
4.2.8. Goto-definition . . . 34
4.2.9. Finding All References . . . 34
4.3. Implementation Guideline for Truffle Languages . . . 34

5. Experience Report **37**
5.1. Limitations of Graal-LSP . . . 37
5.2. Limitations of Our Approach . . . 38

6. Related Work **39**
6.1. Reusing existing tools 41
6.2. Language Workbenches 41
6.3. Run-time type collection 41
6.4. Language-agnostic tagging of symbols 42

7. Conclusion and Future Work **43**
7.1. Future Work 43

A. First Unimportant stuff. **49**

List of Figures

2.1. An exemplary communication of language client and server including initialization, opening and changing a file, and requesting a code completion and goto-definition. 8
2.2. Truffle languages and tools hosted by the Truffle framework on top of the GraalVM 10

4.1. Project structure and dependencies of Graal-LSP 24

List of Tables

3.1. Selected Language Server Protocol (LSP) features and their descriptions 19

List of Listings

1. An example Python code snippet showing a function with two parameters and two local variables to explain completion of globals and locals. 13
2. A Python code snippet showing a property completion example. . . 17

1. An example Python code snippet to explain how to find the best matching node having a source section near the requested completion position. 29

List of Abbreviations

LSP	Language Server Protocol
IDE	Integrated Development Environment
AST	Abstract Syntax Tree
API	Application Programming Interface
VM	Virtual Machine
URI	Uniform Resource Identifier
VSCode	Visual Studio Code
TDD	Test-Driven Development
DSL	Domain Specific Language
IR	Intermediate Representation
SPI	Service Provider Interface
SDK	Software Development Kit

1. Introduction

- language development is challenging task
- what makes a good language: amongst performance, std lib, ecosystem and community, tools (for language developers and users) are important
- who provides the tools? language developers vs. language users
- normally tools are developed for a specific language and cannot be reused or need lots of adaptation
- In the Truffle framework, tooling (instrumentation) is part of the framework and therefore tools are implemented language-agnostic once
- Truffle has already some language-agnostic tools implemented (debugger, profiler, ...) and new ones can be easily built
- Truffle tooling support does not interfere with implementing high-performance languages
- Compared to a full IDE, some tools are missing
- Implementing LSP can provide further IDE-like features
- We propose an approach to build a generic LSP server on top of Truffle with:
  - minimal to zero language runtime performance overhead
  - minimal API changes to Truffle
  - minimal impl and maintenance overhead for language developers
- Benefits for the language developers: get IDE-like tools with minimal effort
- Benefits for language users: can choose from wide range of LSP clients + consistent dev-tools experience for different Truffle languages
- As proof of concept, we have implemented Graal-LSP- A Zero-overhead Polyglot Language Server Implementation for the GraalVM
- We have not implemented all LSP features, but started with the ones which are more easily implementable with Truffle or have are considered as important (code completion)
- ?: focus on dynamically type languages? Or is this focus already included in Truffle (most impls are dynamically typed langs)?

Alternative Introduction

- language development is challenging task
- but we need new languages (or new implementations of old langs) to deal with todays challenges of modern software systems (not only DSLs)
- what makes a good language: amongst performance, std lib, ecosystem and community, tools (for language developers and users) are important
- to ease the development process, language implementation frameworks exist
- but these frameworks either focus on run-time performance (Truffle, RPython) or rich tooling support (language workbenches)
- second: if tooling support is present, it is limited to one or two front-ends (e.g. an Eclipse- or IntelliJ-plugin) the end-users are forced to use -> pain point if programmers need to not only learn new lang, but also new IDE semantics
- we want both
- with new concepts like LSP we have to think about, if we can build a language-agnostic LSP for run-time focused lang impl frameworks
- this would add further IDE-like features out of the box for langs build with these frameworks
- challenging is lang-agnostic IDE-like tooling support for dynamic languages (no static types)
- We propose an approach to build a generic, language-agnostic LSP server on top of run-time focused lang impl frameworks with:
  - minimal to zero language runtime performance overhead
  - minimal API changes
  - minimal impl and maintenance overhead for language developers
- Benefits for the language developers: provide IDE-like tools with minimal effort
- Benefits for language users: can choose from wide range of LSP clients => consistent dev-tools experience for different (Truffle)languages
- to do so, we exploit as much static infos of the framework as we can
- where not possible, we try to learn about our program by observing its execution
- As proof of concept, we have implemented Graal-LSP- A Zero-overhead Polyglot Language Server Implementation for the GraalVM

1.1. Contributions

- Contribution in detail:
  - An approach to build a generic LSP server on top of Truffle
  - An implementation of said approach: Graal-LSP
  - Proposals to extend the Truffle Framework with new interfaces needed to support more LSP features
  - An experience report how well existing Graal/Truffle-implemented languages work with Graal-LSP(with and without implementing the said interface proposals)

1.2. Outline

2. Context

In this chapter we explain terms, concepts and technologies helping to understand our approach in Chapter 3 and the implementation of Graal-LSP in Chapter 4.

2.1. Source Code Editors

A source code editor bundles different features to support developers in their process of writing code. Typically, code editors like vim, Sublime Text, Atom, or Visual Studio Code, focus on editing file-based programming or scripting languages. In addition to the actual text editor view, modern graphical code editors also consist of a file explorer, graphical debugger interface, or build-tools integration, which are often made available as optional extensions for a variety of languages. Popular Integrated Development Environments (IDEs) like Eclipse, Visual Studio, or IntelliJ also have a code editor view as main component, but focus on a richer support of one specific language by providing tailor-made, integrated tools. In the following we will describe selected features of source code editors, which are covered by the LSP described in Section 2.2.

footnote?

footnote?

Navigational Features Source code editors often provide some actions or views to navigate through source code files. Additionally to a pure textual search, there might be a hierarchical outline view of symbols of the current source file or actions to find all definitions or references of a symbol.

Informational Features Another group of features tries to visualization source code related information, e.g. by allowing to hover with the cursor over a function call, to display signature and documentation, or over a variable, to display type information. Furthermore, if the cursor is placed at a symbol in the code, all other locations of the symbol are often highlighted in the current file. For example all read and write accesses of a local variable. If a language parser detects syntax errors or a linter provides some warnings, these information are also displayed in the code editor, typically by underlining the affected source sections.

Code-modifying Features Source code editors often provide some actions to format the whole file or a code snippet according to some formatting rules. Additionally there might be a lightweight refactoring support, like the renaming of a symbol in the current file or all files in the workspace. More sophisticated refactoring tools are rather found in IDE than source code editors.

Code Completion An important feature of source code editors and IDEs is code completion, which is typically integrated seamlessly into the textual editor view. We describe this feature in more detail in the following, because we will use it as running example in our approach in Chapter 3.

Code completion can be triggered manually, e.g. in most editors by pressing `Ctrl+Space`, or automatically when a user types a trigger character. For example in the Java programming language, such a trigger character would be the dot character inserted after a variable. The suggested completion items could therefore be all names of the Java methods or fields declared by the variable's type and super types. If available, a documentation is typically also included in the completion item and for callable objects also a signature. This kind of code completion, where the completion items are depending on the symbol in front of the trigger character, we call in our thesis *completion of properties*. For non-object-oriented languages, *completion of properties* might not be supported or be limited to special language elements, which understand some kind of *property* semantic. *Completion of properties* is often a challenge for languages with untyped symbols, like Python or JavaScript, so that only some IDEs provide a proper code completion system. IDEs like PyCharm typically use static type inference to obtain types for Python expressions.

ref

The other kind of code completion, which you typically get in editors when the caret is placed in an empty line and the user triggers the completion manually, we call *completion of globals and locals*. In this case, the completion items are depending on the surrounding scopes. With *globals* we refer to all the symbols which we can derive from global scopes of languages. With *locals* we analogously refer to all symbols which we can derive from local scopes of languages. In the Python programming language for example, globals are built-in functions or built-in classes. Locals of a function scope are function parameters or local variables. *Completion of globals and locals* might also be triggered automatically, when the user starts typing a symbol, so that matching globals and locals are suggested.

Because code completion is a core feature and is frequently auto-activated while typing, Asaduzzaman has collected requirements of code completion systems [1]. Most of them address the fact, that the completion system should not interrupt the user's flow of code writing.

2.2. Language Server Protocol

In 2016 a group of IT companies, including Microsoft, Red Hat, and Codenvy, started to standardize the Language Server Protocol (LSP) [8]. This protocol describes an approach to tackle the problem of language and tool developers, which need to repeatedly implement language support (e.g. code completion, jump-to-definition and syntax error reporting) for every new code editor, IDE or development tool. The LSP defines a *language server* and a *language client*. The server needs to be implemented only once for every language, providing the language-specific smarts. A development tool implementing a language client can make use of any language server and therefore support multiple languages with minimal effort.

At the date of writing, language servers for more than 50 languages exist[1], including almost all popular programming and scripting languages. Not all implementations support the full feature set defined in the LSP. Usually, these language server implementations support exactly one language, but the LSP allows multiple languages per server.

The LSP version 3.7.0, which is the reference for our approach and implementation of Graal-LSP, specifies requests for typical features of a source code editor: e.g. code and signature completion, navigational features (goto-definition, find all references and implementations, etc.), information on hover, or code formatting. Syntax highlight support is not covered in the protocol and still needs to be handled by development tool specific configuration files or Application Programming Interfaces (APIs).

Client and Server Communication LSP client and server communicate via JSON-RPC[2] requests, responses and notifications. Source code files are referenced by Uniform Resource Identifiers (URIs). A location in the source code is encoded as line and column tuple. During initialization client and server can exchange which features are supported.

Figure 2.1 shows an exemplary communication between server and client. After initialization, the client notifies the server that a file has been opened. The server parses the source code, detects a syntax error and pushes diagnostics to the client. The user fixes the displayed syntax error and the client sends these changes incrementally to the server. After parsing again on the server side, no errors occur, but the server needs to explicitly notify the client about that, so that the client can clear the currently displaying diagnostics. Then, the client sends a request for code completion at a specific source location, typically the caret position. The server obtains completion items depending on the provided context, which might also contain the trigger character, and sends the items to the client. Later, a request to find one or more definitions for a symbol at a position in the source code is sent to the server. The server responds with a list of locations, if found.

LSP4J Our implementation of Graal-LSP uses LSP4J[3] version 0.4.1 as Java binding for the Language Server Protocol. This framework maps the LSP requests and notifications to methods of Java interfaces so that language server implementors do not have to deal with the JSON-RPC communication layer and can make use of Java's type system.

Visual Studio Code We used Microsoft's source code editor Visual Studio Code (VSCode)[4] to create an exemplary language client extension and experiment with Graal-LSP.

Is this important?

[1] https://microsoft.github.io/language-server-protocol/implementors/servers/ Sep. 2018
[2] https://www.jsonrpc.org/specification
[3] https://github.com/eclipse/lsp4j
[4] https://code.visualstudio.com

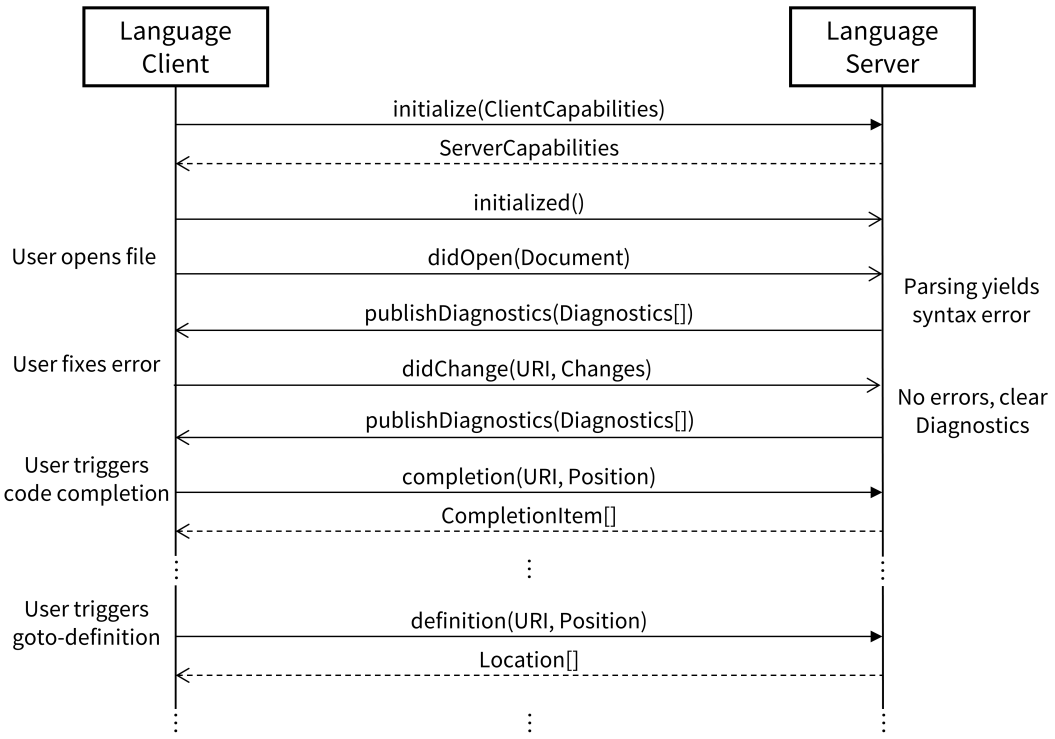


Figure 2.1.: An exemplary communication of language client and server including initialization, opening and changing a file, and requesting a code completion and goto-definition.

2.3. Terms and Concepts

consume in a "Virtual Execution Environment" section?

Abstract Syntax Tree (AST) Parsers for programming languages usually transform plain text source code into an Intermediate Representation (IR) which can be processed by compilers, interpreters or programming environment tools. An AST is such a typical representation which stores structural information of the source code implicitly in its tree structure. Therefore, concrete syntactic elements needed to write parsable code in plain text can be omitted. In general, inner nodes of the AST represent programming language operations and leaf nodes store the values needed for these operations. Depending on the use case, an AST can store additional information in its nodes. For example the lexical source section from which a node was derived. Programming environment tools can use these information to display any kind of errors, warnings or hints directly in the plain text source code. An AST interpreter evaluates an AST by traversing its tree structure and executing the operations defined in the nodes.

Scopes and Frames A scope defines a region of a program in which a set of name bindings is valid, e.g. a local variable name. Most programming languages have a concept of a global (program wide) scope and local scopes. In a lexically scoped language, these local scopes can be obtained at compile-time and are for example defined by the nesting of modules, functions or blocks in the source code. In a dynamically scoped language, these local scopes depend on the call stack, i.e. the nesting of function calls. Today, most programming languages are lexically scoped. At run-time, frames store the actual values of the name bindings of visible scopes.

ref Frame: "Advanced Compiler Design and Implementation" book p.114

expression, statement und co?

2.4. Language Implementation Frameworks

Language Implementation Frameworks help a language developer in the process of creating a new language. The feature set varies immensely, depending on the purpose of the framework. Language workbenches, like Spoofax, Xtext, or MPS, are used to develope and compose Domain Specific Languages (DSLs) together with a corresponding IDE with minimal effort. The focus is therefore on the rich support for tools to easily and efficiently create, compose and work with DSLs. Run-time performance is not the main focus. On the other side of the spectrum, there are frameworks like Truffle [17] or RPython, which focus on implementing high-performance programming languages. Tooling for the developed languages and especially IDE-like tooling, which you get by language workbenches, has not been a focus yet. However, Truffle is an exception, due to its smart integration of an API to build tools.

ref to Fowler, Spoofax, Xtext etc - footnote vs ref?

ref

Truffle and GraalVM Truffle [17] is a language implementation framework written in Java. The language developer uses the framework to write an AST interpreter, which is able to optimize itself during interpretation [20]. To achieve that, Truffle uses run-time type information to rewrite AST nodes. Callable elements of a language, e.g. functions or methods, are modeled as root nodes. They can be executed by providing a frame object. A node can have a source section attribute which encodes the range in the source code file it relates to. How source code is parsed and the AST is structured leaves Truffle in the responsibility of the language implementation.

Truffle has built-in support for developing language-agnostic tools by providing an instrumentation API [12]. It is designed to work well with the optimization techniques for language implementations so that instrumentation can be done with nearly zero-overhead [13]. Instruments can be installed, like languages, by putting them on the Java class path to load them via Java's Service Provider Interface (SPI) . Instruments can register and unregister listeners dynamically for different events to observe behavior of the language interpreters. For example, a listener can be notified when a new source or source section is loaded, a statement is going to be executed or an expression will return a value. It is also possible to inject behavior into the language interpreters by attaching node factories. It can be used for example, to alter the current execution flow and let a function return a different value when

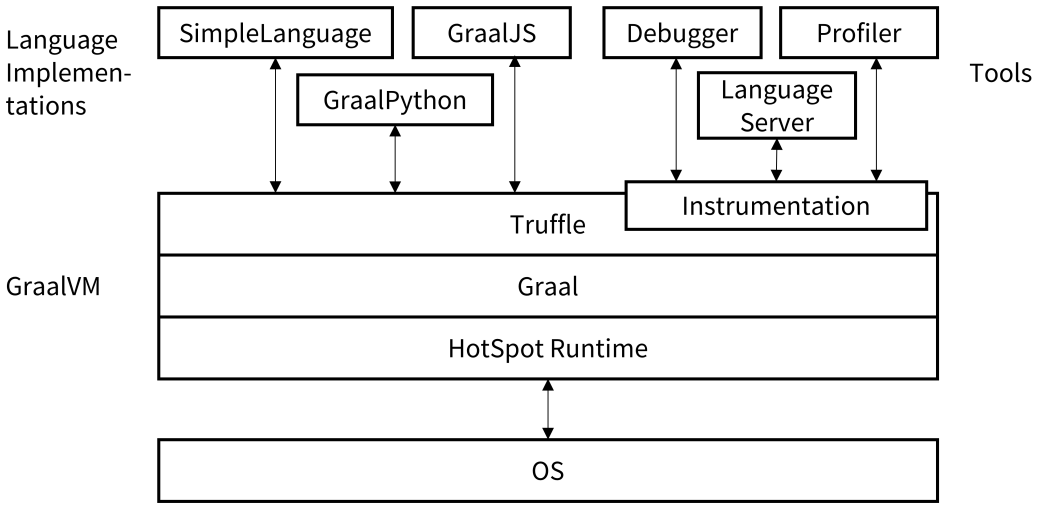


Figure 2.2.: Truffle languages and tools hosted by the Truffle framework on top of the GraalVM

called under certain conditions. This instrumentation enables building powerful tools like debuggers or profilers which need to be implemented only once and can then be used for all Truffle languages. To control which AST nodes of a language are available for instrumentation, language implementors need to mark nodes as instrumentable. A tagging mechanism allows adding meta-information, like "this node represents a statement in my language", which can be used by instruments to filter for specific nodes. For example, a debugger can facilitate these information to implement stepping. The tools available at the date of writing are mostly run-time tools like debuggers, dynamic profilers and language extensions [13].

Cross-language interoperability is another feature of the Truffle framework [4]. Language implementations can be hosted by the Truffle framework in a multi-language run-time and communicate via language-independent messages. Foreign language objects can be accessed directly without marshalling.

To get high-performance language implementations, Truffle uses the language-independent just-in-time compiler named Graal. It has its roots in the research project of the Maxine VM [16]. Among other optimizations, Graal applies partial evaluation to compile stable AST subtrees into optimized machine code [18, 3]. These optimizations do not stop at language boundaries, so that the access of foreign language objects can also be optimized.

cite or footnote?

GraalVM[5] is the umbrella term for a HotSpot VM which is bundled with the Truffle language implementation and instrumentation framework and the Graal compiler, building a high-performance polyglot VM for applications written in a Truffle-implemented language [19]. Figure 2.2 visualizes that architecture.

[5] https://www.graalvm.org/

RPython

- bytecode interpreter
- type inference [7]
- http://stefan-marr.de/papers/oopsla-marr-ducasse-meta-tracing-vs-partial-evaluation/

3. Approach

In Chapter 1 we described the problem, that run-time performance-focused language implementation frameworks lack IDE-like tooling support for the developed languages. Therefore, we suggest to build a language-agnostic language server on top of these kind of frameworks to start closing the tooling gap. Because most of the currently implemented languages of frameworks like RPython or Truffle are dynamic and the support of features like code completion is especially difficult for these kind of languages, we focus on dynamic and dynamically typed language implementations in our approach.

Typically, a language server is developed for one specific language only and the server implementation can exploit any kind of language-specific information. To implement a language-agnostic language server, we have to build upon the abstract concepts and APIs of language implementation frameworks, which apply to all languages developed in these frameworks. In the following, we use the example of code completion to described our approach. Fist of all, we show which kind of data is needed to respond to the *completion* request of LSP. Then, we suggest how to obtain the needed information in a language-agnostic way and where the language implementations need to hook into.

3.1. Completion of Globals and Locals

As described in Section 2.1, there are two kinds of code completions we have to distinguish. We start with the Python code snippet in listing 1 to explain how to support code completion of globals and locals.

Listing 1: An example Python code snippet showing a function with two parameters and two local variables to explain completion of globals and locals.

```
1  def foo(param1, param2):
2      x = 42
3
4      y = param1 + param2
5
6      print(x * y)
```

Imagine, the listed Python code snippet is the content of a file which was opened in an LSP-supporting source code editor and the language client implementation has already reported the opening of the file using the *didOpen* notification to our imaginary language server. This notification includes the content and the URI of the file. Now, a *completion* request is sent to the server for line 3 at character offset 4. This means, the completion is requested in the empty line between the assignments of the

two local variables `x` and `y`. We would expect a completion result to include items for all Python globals (e.g. built-in functions) and the locals `param1`, `param2`, and `x`. `y` should not be included, because a value is assigned to it not until line 4. Additionally to a display name, a LSP completion item can optionally contain a completion kind (e.g. variable, function, or class), a detail text for signature or type information, and a documentation. To provide these items, we suggest the following steps.

1. Parse the source code into a language-agnostic IR of the framework
2. Map line and column of the completion request to the IR
3. Derive lexical scopes from the mapped IR-element
4. Build completion items by gathering additional information for the obtained symbols of the scopes

In the first step of parsing source code into an IR of the framework, we can reuse the existing parser, which the language developer has to provide anyway. This parser will typically be syntax-error intolerant, so that we can support code completion only if the source code is parsable.

The second step requires, that selected elements of the IR contain references to their lexical representations in the source code, or at least a process to obtain these references afterwards. If we assume an AST as IR, relevant nodes could store start- and end-offsets of characters for the parsed source code. Then, the IR elements need to be searched for an element which maps best to the position of the completion request. Which one that should be, depends on the IR and the framework. A tagging mechanism could help to mark special IR-elements in a language-agnostic way, to allow filtering.

In the third step, we assume that the language implementation framework has a concept of scopes for the language implementations. For now, we are interested in lexical scopes only. Typically, there are some language built-ins which are always available, independent of the current context. They can be grouped in a global scope. The IR-element, matching the completion request position, can be used to derive local scopes together with symbols valid in these scopes. If we take an AST as IR example again, local scopes can be obtained by traversing the AST. All scope-building language elements, like functions, classes or blocks, can be collected along the way, starting at the node, which matches the completion request position, up to the root.

Finally, we create the completion items based on the symbols of the scopes. Depending on the meta-information available for each symbol, we can add a completion kind, signature information for callables, or a documentation text. Due to the dynamic nature of our targeted languages, these information will be typically not available at compile-time.

3.2. Obtaining and Using Run-time Information

A detailed code completion for globals, locals, and properties of language elements is often implemented in IDEs using type information. For programming languages with typed symbols, like Java, the required information can be easily derived from by parsing source code. The situation for languages with untyped symbols, like

Python, is different, because the type depends on the associated run-time object. There are language-specific approaches to infer types of variables for such languages , but these approaches ... .

ref

In this section, we describe an approach on how to enhance code completion for locals using run-time information. This approach will also build the basis for *completion of properties* described in Section 3.3.

far from perfect, difficult to built in a language-agnostic way, ...

Approach and Challenges The basic idea to get run-time values for locals at a specific position in the source code, is to execute that source code while observing the execution to collect run-time information at the desired position. This approach implies some preconditions. Language implementation frameworks need to provide a possibility to execute a program until a specific source code position is reached and then to inspect the current run-time state. This is already possible, if the framework provides a debugger for the developed languages. If present, we can reuse the same approach as used by the debugger to halt at a breakpoint and obtain local symbols and their corresponding run-time values from the active frame. To create the actual completion items, the symbols can be used as display names. Details, completion kind and documentation can be determined from the run-time values if the framework supports accessing these meta-information by querying the language implementations.

Unfortunately, executing arbitrary source code requires to think about how to deal with unexpected and unwanted side-effects during execution, the possibility of a long running or non-terminating execution, or the fact that the desired position might not be reached by the execution flow.

Unreachable Source Sections Depending on the source code, there might be source sections which are unreachable by the normal execution flow of the current source code file. In our opinion, this is the normal case for file-based programming languages. You typically split your program's source code into multiple files, having a limited set of files, which define execution entry points, which then import elements of the other files.

One of these entry points are unit tests. Programmers typically write them to ensure the desired behavior of their programs and avoid regression of bugs. Therefore, unit tests should cover most parts of the source code. Furthermore, most unit tests have a short execution time and are supposed to be free of unwanted side-effects – or at least have a clean-up routine included. The systematic execution of unit tests is therefore a promising approach to get run-time information for code completion enhancement. This idea is inspired by *type harvesting* [5], a practical approach to obtaining typing information in dynamic programming languages by exploiting unit tests. We only have to provide programmers a way to tell our language server how to execute the tests. This could be done via a separate configuration file or via a special comment in the first or last lines of a source code file, which encodes a path to a runnable script.

Using the described approach, we have to care about unsaved changes. If we execute unit tests, we have to ensure that they do not import files from the file

system, if our server has newer, but yet unsaved versions. The LSP specifies no request sent from server to client to trigger saving files and we do not want to write the changes to the file system from server-side. Therefore, we suggest to redirect file accesses of files with unsaved changes to (virtual) files which contain the changes. This could be realized by installing a custom file system, if the framework supports that. Otherwise, an enhanced code completion using run-time objects is only possible after saving all changes actively at client-side.

explain in background?

Unit tests might not be available, especially if you start writing a small script or do not follow Test-Driven Development (TDD) principles . Therefore we suggest as fallback to try executing the current source code, in which the code completion was triggered, if no path to an execution entry point is defined.

Sandboxing the Execution Environment Users would not expect that the source code, they are currently working on, will be executed when they trigger code completion. Therefore we suggest to create a sandboxed execution environment, when the execution of arbitrary source code is triggered implicitly. To create such a sandbox, the framework needs to provide an abstraction layer, which allows to control all I/O of the implemented languages. If supported, the most radical approach would be to forbid all I/O in our sandboxed environment. Unfortunately, our approach of using run-time information for code completion would thereby be limited to theoretical examples only. Therefore, we suggest to allow at least some operations. For standard I/O, we suggest to allow any kind of input, but redirect or disallow all output. For file I/O, we suggest to simulate a read-only file system or, if supported, install a custom copy-on-write file system, which copies each file before modification to a sandbox and which redirects further read and writes to that copies. For network I/O, we cannot make any assumptions concerning side-effects and suggest therefore to disallow all network I/O.

The situation is different, if users explicitly allow side-effects. This might be the case if they have defined an execution entry point, like a unit testing script, which performs only known and uncritical side-effects. Therefore, we suggest to provide a configuration option, to tell our language server, if sandboxing shall be deactivated when collection run-time information for code completion.

Long Running or Non-terminating Executions If the completion request takes to long to return a result, users might become impatient and distracted. Unfortunately, the execution of arbitrary source code might take an unpredictable amount of time or might not finish at all, for example due to an endless loop or blocking operations. Furthermore, the LSP does not support responding completion results incrementally. Therefore, the first thing we suggest is to explicitly stop execution when the execution flow has reached the desired position in the source code and we have collected the needed run-time objects to build our completion items. Additionally, we suggest to add a general timeout to ensure an upper bound of execution time. In case of a timeout, no run-time information can be provided to enhance our code completion, but an endless execution can be avoided.

Caching Run-time Information Executing a script or unit tests for every completion request might yield no results due to long running executions and timeouts. Therefore, we suggest to collect run-time information for the whole source code once and cache these information for later completion requests. This collection run should be triggered manually by the users, e.g. by using custom commands, which the server can register during LSP initialization. These commands can then be made accessible at client-side via the *codeLense* request. This way, users can trigger the renewal of the cache manually, when they considered it reasonable.

As a language-agnostic data structure for the cache, a mapping from lexical source sections of IR-elements to collected run-time objects can be used. To get a look-up key when a completion request arrives, we can use again the approach of finding an IR-element near the completion request's position, to get the IR-element's lexical source sections. If the look-up succeeds, we can use the cached information to create the completion items.

The challenge of using such a lexical mapping is, that every time a *didChange* notification arrives, one or more sections of the source code will change. Depending on the change, the mappings in the cache might become invalid, but others might still be useful. We assume, that most edits will affect only mappings of the current local scope and in that scope only the edited line and following, so that all other mappings might be migrated. If the mapping keys reference source positions only via line number and character offset in that line, the migration could be done by simply adjusting the line number, if the changes add or remove one or more lines. Of course, we can only assume that the migrated mappings are still valid. The user has to trigger a new collection run manually when, for example, the execution flow is changed, or variables are renamed.

3.3. Completion of Properties

The second kind of code completion we want to support is the *completion of properties*.

listing 2 shows a Python code snippet, containing a class `Natural` with an instance variable `n` and a method `next`. In the last line, we assume that the dot was the last character inserted and a code completion request has been triggered. We expect, that a completion result includes `n` and `next` as properties of the object referenced by the symbol `natural`. Other inherited properties, which all Python objects have, might also be included.

Listing 2: A Python code snippet showing a property completion example.

```
1  class Natural:
2      n = 2
3
4      def next(self):
5          r = self.n
6          self.n = self.n + 1
7          return r
8
9  natural = Natural()
10 natural.
```

To support this kind of code completion, we can build on the approach of obtaining and using run-time information described in the previous section. But there are some additional challenges which we have to solve. First of all, we have to detect that an object property completion shall be triggered instead of a completion of globals and locals. Second, the source code will typically contain a syntax error, when this kind of completion request is sent. Third, we have to identify the IR-element for which the completion of properties is desired. Fourth, we have to obtain the actual object associated with that IR-element, which might be available only at run-time. Finally, we need to get the object's properties to create the completion items.

Detecting an Object Property Completion As prerequisite for completion of properties, we need to be able to query a language implementation for its completion trigger characters. When a completion request arrives, we can then distinguish the two kinds of completion by comparing one or multiple characters directly in front of the completion request position with the trigger characters of the current language. If there is a match, we try a property completion.

To support auto-activation of code completion, the trigger characters need to be sent to the client as part of the *initialize* response.

Finding the Desired Node To find the IR-element in front of the completion trigger character, the source code needs to be parsable. We assume, that this is not always the case when a completion is triggered, because an inserted completion character might cause a syntax error. Therefore, we suggest to revert the last textual change which was sent from the client via the *didChange* notification and then try to parse the reverted source code again. Translated to our Python example in listing 2, the source code shown is no valid Python source code because of the last dot. Removing this dot makes the code parsable again.

add remark: approach only possible for languages which support "expression statements"? <- how can we name that? For example in Java this would not work.

Now, we can search for a matching IR-element in front of the removed completion trigger character position, by checking its lexical source section. However, there might be multiple IR-elements whose lexical source sections end before the removed trigger character position. Here it would be useful again, to have tagged IR-elements, so that we can filter for example for expressionsonly.

is not fitting. Defer to impl?

Obtaining the Actual Properties The next step is getting the corresponding run-time object by evaluating the matching IR-element. In our Python example, we would evaluate an IR representing a variable read for the symbol `natural`. We can use the already described approach for obtaining run-time information for locals, to execute the source code until the desired IR-element is reached. Then, the framework needs to provide an API, so that we can intercept the result of the IR-element evaluation. Having the run-time object, we need the support of the framework to query the language implementation for properties of that object, together with further meta-information of the properties, to add details, kind, and documentation for each completion item.

We suggest some optimizations to avoid executing source code for every property completion request. First of all we can optimize property completion for literals,

Table 3.1.: Selected LSP features and their descriptions

Request/Notification	Description
signatureHelp	Request to find signature information (e.g. parameter names and details for a function) at caret position. Is triggered via registered signatureHelp trigger characters.
hover	Request for information at caret-hovering position.
documentSymbol	Request for symbols of a specific source file (function names, variable names, module names, etc.).
symbol	Like documentSymbol, but for all files in the client workspace.
definition	Goto-definition request for functions, variables, etc.
references	Request to find all references of a symbol.
documentHighlight	Request to highlight all ranges related to a symbol (e.g. variable reads and writes).

if the framework supports identifying IR-elements as literal representations. The literal value is typically encoded in or referenced by the IR-element. If we can access this value, we can directly query for its properties.

If the IR-element is not representing a literal, we can check as next step if there are already cached run-time information available. If the framework and the language implementation support in-line evaluation, we can evaluate the IR-element in the context of the cached information.

hier noch mehr erklären - Beispiel geben mit frame usw oder dann in Impl erläutern?

If using cached run-time information returns no result, we need to execute unit tests or the current source code to get the required run-time object.

If there is still no result, because the source position of the completion request might not be reachable or a timeout occurred, we can use a last fallback. We suggest to evaluate the IR-element in the global scope. This will at least provide a property completion for globals and context-independent expressions.

Sum up / Überleitung

3.4. Transferring the Approach to Other Requests

The approach of supporting code completion, described in the previous sections, includes parts, which can be used to support further LSP requests. Table 3.1 shows selected LSP requests, for which we will describe the approach transfer in the following.

Zusammenfassung, was wiederverwendet werden kann: run-to-section + static search for tagged IR-elements + IR-element lexical source section

Signature Help The *signatureHelp* request is auto-activated at client-side, like code completion, if the server includes trigger characters for signature help in the *initialize* response. A *signatureHelp* response consists of a list of signatures, because there might be different variants, if the language implementation supports function overloading. Furthermore, the response includes two indices, telling which signature and which parameter should be shown in detail in the UI. Every signature in the list consists

of a label, a list of parameters, and an optional documentation. A parameter also consists of label and documentation.

To collect all the needed information, we suggest the following steps.

1. Find a matching IR-element at the requested position
2. Execute the source code until the IR-element is reached
3. Intercept the callable run-time object represented by the IR-element
4. Query the callable for label, parameters and documentation

The first to third steps equal those for code completion of properties. We only need to search for an IR-element which represents a callable, e.g. via a tag. A challenge is, that the signature help should be displayed from the time, the trigger character is inserted, until the user has completed the signature. During that process, the source code might contain syntax errors, so that we have no IR-elements. Returning an empty response will end the displayed signature help. Therefore this request might be supported only partial.

Having obtained a callable, the last step implies, that we can query the language implementation for information about that callable. Among all parameter names and their documentation, we need for the label a formatted signature, which depends on the syntax of the language. If this is not supported by the framework, but we can get at least access to a list of parameter names, we suggest to use the widely used Python-like syntax `callableName(param1, param2, paramN)` as signature label.

Hover Information With the already described approaches, we can support the *hover* request for variables and callables with minimal effort, if there are cached run-time information available. The steps to build the hover response are as follows.

Punkt am Ende?

1. Find a matching IR-element for a callable or variable at the requested position
2. Use the cached run-time information to perform an in-line evaluation
3. Create a hover response from the evaluation result

The first step requires again some kind of tagging of IR-elements. If we have found an IR-element representing a variable or callable, we can perform an in-line evaluation as described for code completion. Using the evaluation result, we can build the hover response, which consists of plaintext or markup content.

Document Symbols The *documentSymbol* response items consist of a label, range, symbol kind, and container name. To provide these information, we suggest the following steps.

1. Find all IR-elements representing symbol declarations in a source code file
2. Query these IR-elements for the requested information
3. Create a response containing a list of symbol information

As for the *hover* request, the first step requires some kind of tagging, to filter for the required IR-elements. The second step requires, that we can obtain label and range, and optionally symbol kind and container name. The label and range can be

derived from the IR-element and its lexical source section. Supporting a symbol kind depends on the meta-information available for IR-elements. The container name could be derived from the names of the surrounding lexical scopes.

Workspace Symbols The *symbol* request is the union of *documentSymbol* requests for all files in the workspace. Therefore, we can simply apply the approach explained in the previous section to support the workspace-wide *symbol* request.

Document Highlight The *documentHighlight* response consists of a list of ranges in the source code. Each range can have an associated kind (`Text`, `Read`, or `Write`). To support highlighting of variables, we suggest the following steps.

1. Find a matching IR-element representing a variable at the requested position
2. Search for all other IR-elements representing the same variable in the current source code
3. Create a response containing the list of ranges and kinds

The first step requires again to search and filter for an IR-element at the requested position, which represents a read- or write- access of a variable. For the second step, we suggest to search in the surrounding lexical scopes for all other IR-elements representing an access to the same variable. The required range can be obtained from the lexical source section of the IR-element. The kind can be mapped from the access kind of the variable.

Goto-definition

- response: Location[]
- local variables and callables
- local variables: like *documentHighlight* -> interpret first write as definition
- callables: like code completion -> run to section + query callable run-time object for its source section
- static fallback: like *documentSymbol*, find all definitions of symbols in the workspace and match callable name. Might yield false-positives.

Find All References

- response: Location[]
- local variables and callables
- local variables: like *documentHighlight*
- callables: find other IR-elements tagged as calls with same identifier

4. Implementation

In this chapter, we describe how we have implemented our prototype Graal-LSP, based on the approach described in Chapter 3. Graal-LSP uses Truffle [17] as underlying language implementation framework and exploits Truffle's instrumentation environment to access the required information in a language-agnostic way. We also implemented small Truffle API extensions to support more LSP features. As LSP Java wrapper we used LSP4J . Transportation for JSON-RPC is implemented using TCP sockets. In the following sections, we start by describing how we have implemented common features required for most LSP requests and then explain in detail the implementation of selected LSP requests in Graal-LSP.

name a version?

4.1. Prerequisites to Answer LSP Requests with Truffle

- our approach has some requirements on the language impl framework
- influences project structure and dependencies
- server init
- things common for most requests:
  - parsing + diagnostics
  - mapping caret to IR
  - obtaining run-time information

4.1.1. Project Structure and Dependencies

To facilitate Truffle instrumentation, we have to implement the interface `TruffleInstrument`, which is part of the Truffle API. Instruments are loaded via Java's SPI and the framework calls their implemented `onCreate(Env)` method during initialization, to pass an instance of a Truffle instrumentation environment. This environment has a language-agnostic API to parse source code, interact with loaded languages, and observe and inject behavior into the AST interpreters. Most LSP features can be mapped to that API.

As described in Section 3.3, our approach requires to sandbox the execution of guest languages and redirect file accesses. Unfortunately, the Truffle instrumentation environment provides no API to support that requirement. However, there is a separate Graal Software Development Kit (SDK) to embed and run code from guest languages in JVM-based host applications. It allows to create and configure a Polyglot `Engine` and Polyglot `Context` instances. An `Engine` provides access to the installed languages and instruments. A `Context` instance allows to evaluate code of its associated languages and represents the global run-time state. Furthermore, it

context?

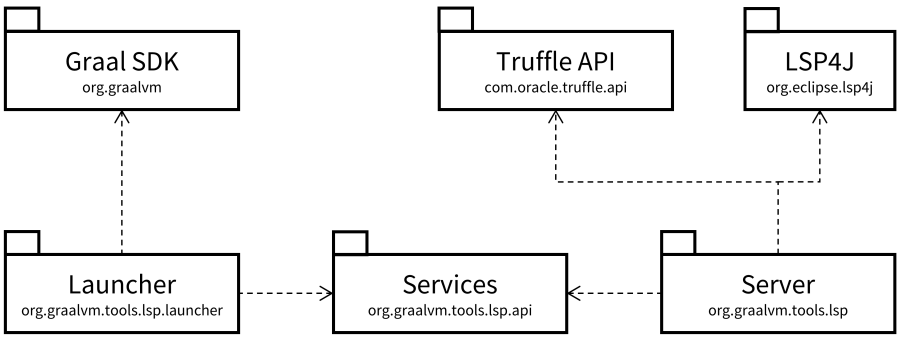


Figure 4.1.: Project structure and dependencies of Graal-LSP

supports to set a custom file system. Multiple `Context` instances can share the same `Engine` instance and thereby all its registered instruments.

Therefore we decided to create three Java projects. The first contains a separate launcher which handles the whole bootstrapping and enables the creation of sandboxed `Context` instances using the Graal SDK. The actual language server is implemented as Truffle instrument using the Truffle API in the second project. During bootstrapping, out instrument will register some services which enable the launcher for example to explicitly start the language server. These service interfaces are part of a third project on which the others depend. Figure 4.1 visualizes the resulting project structure and dependencies.

4.1.2. Threads and Context Activation

Every operation performed on the Truffle instrumentation environment requires, that the current Thread has entered a Polyglot `Context`. Because some Truffle language implementations do not support multi-threaded access, we decided to have only one worker Thread, which performs operations on the Truffle instrumentation environment. This Thread explicitly enters a `Context` instance at start-up once. We use a `java.util.concurrent.ExecutorService` instance to submit tasks to that worker Thread. Incoming LSP requests are accepted by a Thread of the LSP4J framework. To avoid blocking this Thread, it only wraps a request into a task and submits it to the executor service. A separate Thread picked from a dynamic pool will await the result of the context-entered worker Thread using a `Future`. It will send the prepared response to the client and handle occurred errors by sending diagnostics in addition.

figure showing a typical requests flowing through the different Threads

Reusing Contexts

das muss irgendwo noch hin -> optimization: reuse Contexts, new Context erst wenn coverage analysis is triggered. Aber passt hier auch noch nicht so richtig. wohin?

4.1.3. Language Server Initialization

The first request sent from the client to the server is the *initialize* request. To support auto-activation of code completion and signature help, the response has to include trigger characters. A language-specific language server implementation can provide these characters statically, because the set of supported languages is well known. In our language-agnostic implementation, where Truffle languages can be installed dynamically, we have to collect these characters for all available languages during server initialization. Using the Truffle instrumentation environment, we can get a list of all registered languages, but there is no API to obtain trigger characters for code completion and signature help from a language. Therefore, we have extended `TruffleInstrument.Env` and `TruffleLanguage` with methods to retrieve these characters.

4.1.4. Parsing Source Code

To parse source code in Truffle, we need to build a `Source` object, containing the language ID and a reference to the actual source code, typically via a file. Then, we can us the instrumentation environment to call `parse(Source source, String... argumentNames)`. In-line parsing is also supported via the method `parseInline(Source source, Node node, MaterializedFrame frame)`.

The LSP requests *didOpen* and *didChange* both contain an URI pointing to the concerning source file. The *didOpen* request also contains the whole file content and a language ID. Some language clients send a media type as language ID, therefore we test if the ID belongs to a registered language via the instrumentation environment. If not found, we try to match it with the media types defined by each language.

In our implementation, we configured, that the *didChange* request contains only incremental changes. Therefore we have to store the editor content together with the language ID at server-side and apply all incoming changes to keep track of the client-side state. Therefore we created the class `TextDocumentSurrogate`. An instance is created for each *didOpen* request, to store all data corresponding to a source file at client-side.

The *initialize* request can contain a workspace URI. If present, we parse all files in the workspace folder and its subfolders once, which match one of the media types of the installed Truffle languages.

In case of a parsing error, we can notify the client using the *publishDiagnostics* notification.

Diagnostics Truffle languages typically throw a `TruffleException` in case of a parsing error. The parsers are not designed to be syntax error tolerant and will stop parsing at the first syntax error. A `TruffleException` can store an AST node as location where the exception occurred. If available, we map the node's source section to line and column of the original source code and send a *publishDiagnostics* notification to the client. If the location is not available, we use the first column

in the first line as reference for the notification. The error message needed for the notification is also obtained from the `TruffleException`.

4.1.5. Mapping Requested Source Position to AST Node

Most LSP requests are context-sensitive and include therefore line and column as reference. Line numbers are zero-based and a newline character at the end of a file counts a separate line. Truffle's `Source` objects use one-based line numbers and do not count an empty line at the end. Therefore we have to convert the request parameters line and column to match the Truffle sources.

As described in our approach, we have to map line and column of the LSP requests to IR-elements of a language, to use them for further querying and processing. The Truffle instrumentation environment can for example obtain lexical scopes from their AST nodes. Therefore, we are either interested in an AST node having a source section which contains the requested position (e.g. *hover*, *definition*, and other requests). Or in a node having a source section which is near the requested position (*completion of globals and locals*). To search for these nodes, we can use the Truffle instrumentation environment to access all already loaded source sections and their corresponding nodes by attaching a `LoadSourceSectionListener` with a filter for the current `Source` and the `includeExistingSourceSections` parameter set to `true`. Our implemented listener collects the node with shortest source section containing the requested position as `containingNode`, the one with shortest source section ending before the requested position as `previousNode`, and the one with the longest source section beginning after the requested position as `nextNode`.

This selection process requires that language implementations support a consistent source section hierarchy. That is, if a node has a source section and is a child of another node, then the source section of the child is fully contained in all source sections of its direct parent or indirect parents (ignoring parents without source sections). If the assumption holds, we can take the `containingNode` as best match for further processing for requests like *hover*, *definition*, or *references*. For *completion of globals and locals*, the selection process is more complicated and will be explained in Section 4.2.1.

As another prerequisite, we assume that source code is eagerly parsed when calling `parse(Source source, String... argumentNames)` and not lazily when the code is going to be executed. Otherwise, no nodes and source sections will be available via the instrumentation environment.

4.1.6. Obtaining Run-time Information

In Section 3.2 we explained an approach to provide a richer code completion for locals and support further LSP requests by observing the executing of source code. We describe in the following how we have implemented most of that approach in Graal-LSP.

execution of source code - or- evaluation of source code?

Observing the Execution The instrumentation environment provides the method `attachExecutionEventListener(SourceSectionFilter, ExecutionEventListener)` to observe executions of the current `Engine`. The `ExecutionEventListener` will be notified immediately before and after an instrumented node is executed. The `SourceSectionFilter` allows to filter for which source sections or node types the listener shall be activated.

@Tim/Fabio how to handle that?

Unreachable Source Sections As described in our approach, we need to define an execution entry point, e.g. a path to a script which triggers unit tests, to cover a maximum of source code. For now, we have implemented to search for the string `"RUN_SCRIPT_PATH:"` in the first line of the source code and interpret the rest of the line as file path. We create a `Source` object for that file and parse it using the instrumentation environment's `parse(Source source, String... argumentNames)` method. The return value is a `CallTarget` which we can execute. As further proposed in the approach, we fall back to executing the current source code, in which the request was triggered, if no `"RUN_SCRIPT_PATH:"` string is found.

To handle unsaved changes, we have suggested in our approach to install a custom file system, which intercepts read accesses to files and delegates them to a virtual file system which includes the changes. By using the Graal SDK as described in Section 4.1.1, we can create new `Context` instances and can use the method `Context.Builder.fileSystem(FileSystem fileSystem)` to install our own file system. We have implemented such a file system in the class `LSPFileSystem`. It holds a callback to our language server, to delegate read accesses, if the file is known by the language server.

Sandboxing the Execution Environment To support basic sandboxing of the file system, we made our `LSPFileSystem` read-only. At the date of writing, Truffle supports a restriction of file I/O only. Therefore, network and system I/O is still unrestricted in our implementation.

Long Running or Non-terminating Executions If we observe the execution using an `ExecutionEventListener`, we can stop when we have collected required run-time information. Therefore, we simply throw a special `ThreadDeath` exception, containing the collected run-time object. This exception is also used by other Truffle tools to stop or unwind the current execution. We catch this exception in our language server implementation, to retrieve the run-time object. In our approach, we have also suggested to cancel the execution in case of a timeout. Due to time constraints, we have not implemented this feature yet.

Caching Run-time Information To cache run-time information, we use the instrumentation environment to attach an `ExecutionEventNodeFactory`. We let the factory create special `ExecutionEventNode` instances, which capture a snapshot of the passed frame at the first call of `onEnter(VirtualFrame frame)`. As data structure, we use a `Map` associating `MutableSourceSection` with `CoverageData`. Every `TextDocumentSurrogate` instance holds a reference to such a `Map`. A `MutableSourceSection`

is a light-weight references to a node's source section by storing only line, and column for start and end. A `CoverageData` instance stores a `MaterializedFrame` and the `ExecutionEventNode` which captured the data.

name limitation here, that captured frame snapshot is no deep copy?

The collection of run-time information can be started manually by the user by activating a command, which we provide via the *codeLense* request in the first source code line.

To avoid invalidation of all our mappings, when a *didChange* notification arrives, we use a simple migration strategy for our `MutableSourceSection` instances.

1. If a cached section's end line is smaller than the start line of the change, we do nothing.
2. If a cached section's start line is greater than the end line of the change, we check if the change adds or removes lines. If so, we adjust start and end line of the cached section by that difference.
3. If the range of the change is fully included in the cached section, we adjust only the end line of the cached section by the difference of added or removed lines.
4. In all other cases where start or end lines of cached section and change overlap, we assume that the mapping is not valid anymore.

This way, we can migrate cached run-time information mappings for simple changes. But we can only assume that the adjusted mappings are still valid. The user has to trigger a new collection run manually, if a change alters execution flow, names of variables, or types of variables.

4.2. Implementing LSP Request Handlers

In this section we will explain how we have facilitated the Truffle instrumentation environment to support selected LSP requests. For most LSP requests we have implemented a separate Java class. They inherit all from `AbstractRequestHandler`.

class diagram?

4.2.1. Completion of Globals and Locals

To implement the code completion approach described in Section 3.1, we can mostly use already existing methods from the Truffle instrumentation environment. The method `findTopScopes(String languageId)` provides global scopes of a language and the method `findLocalScopes(Node node, Frame frame)` provides the local scopes. To use the latter, we have to find a node having a source section near the requested completion position.

Finding the Nearest Node We explain the difficulty of finding that node by using the following Python code snippet.

label counter wrong?

listing 1 shows a function definition `foo`, containing two parameters and two local variables in its scope and a function `bar` without definition of locals. Imagine that a

Listing 1: An example Python code snippet to explain how to find the best matching node having a source section near the requested completion position.

```
1 def foo(param1, param2):
2     x = 42
3
4     y = param1 + param2
5
6
7 def bar():
8     return 42
```

completion request is triggered in line[1] 3 at character offset 4. As already mentioned, Truffle cannot derive scopes directly from the caret position, but from a node. The difficulty is, that the derived scopes will contain variables valid *at* that node. This means translated to our Python example, that if we use the node representing `y = param1 + param2` in line 4 to derive local scopes from, the scope of function `foo` will contain variable `x` but not variable `y`. One could say, that we get scopes with all locals valid before the node's source section starts. Imagine another code completion is requested in line 5 at character offset 4. The problem is now, that there is no node which will provide us with a scope containing the variable `y`, although we are actually still in the function scope of `foo` and the completion is triggered one line after a value was assigned to `y`.

IntelliJ code completion uses a dummy reference, if there is none at the caret - should we name that here, that we cannot do that?

As described in Section 4.1.5, we can use the Truffle instrumentation environment to find a `containingNode`, `nextNode`, and `previousNode` based on the requested position. To decide, which one will most likely deliver the expected locals for the completion items, we use the following best-effort algorithm.

1. If we cannot find a node having a source section which contains the requested position, we are not in a local scope. No completion of locals necessary. (Example: line 6, offset 0)
2. If contained, we pick `nextNode` if it is also an (indirect) child of `containingNode`. (Example: line 3, offset 4, pick the node representing `y = param1 + param2`)
3. If not found, we pick `previousNode` if it is also an (indirect) child of `containingNode`. (Example: line 5, offset 4, pick the node representing `y = param1 + param2`)
4. If still not found, we use `containingNode`. (Example: line 4, offset 10, pick the node representing `param1`)

Querying Properties of Scope Values Calling `findTopScopes(String languageId)` or `findLocalScopes(Node node, Frame frame)` returns a collection of `Scope` objects. A `Scope` can be lexical and dynamic. `Scope.getVariables()` returns a `TruffleObject`, which can be queried in a language-agnostic way via sending messages. Using a `KEYS` and a `READ` message, we can obtain key-value pairs, where the key is the name of the declared variable and the value is again a `TruffleObject` representing the assigned value.

[1] LSP uses zero-based line numbers. For simplicity, we will always use one-based line numbers in our examples.

For locals, we can only obtain lexical scopes when calling `findLocalScopes(Node node, Frame frame)` with `frame=null`. The returned local `Scope` objects might contain declared variable names, but no values will be available. Therefore, we create completion items for locals by using the variable name as display name, the scope name as detail text, and `CompletionItemKind.VARIABLE` as default completion item kind.

For globals, we can therefore create completion items with more details, because a value will typically be available. We can query the value, which is a `TruffleObject` again, for meta-information by sending the following messages. To determine a completion item kind, we use `IS_INSTANTIABLE` and `IS_EXECUTABLE`. If the first returns `true`, we use `CompletionItemKind.CLASS`. If the second returns `true`, we use `CompletionItemKind.FUNCTION`. Default is again `CompletionItemKind.VARIABLE`. To add the name of the value's meta-object to the detail text of the completion item, we use the Truffle instrumentation environment to call `findMetaObject(LanguageInfo language, Object value)`.

We added two more custom messages (`GetDocumentation` and `GetSignature`) to support querying a `TruffleObject` for documentation and signature of callables. If a language implementation does not support `GetDocumentation`, we try if we can get the source code of the callable definition as documentation for the completion item, using the method `findSourceLocation(LanguageInfo language, Object value)` of the instrumentation environment.

Using Cached Run-time Information If cached run-time information are available when a completion for globals and locals is triggered, we use the nearest node's source section to search for a cached `CoverageData` object. If found, we can call `findLocalScopes(Node node, Frame frame)` with the cached frame. Now, the resulting `Scope` objects will also give access to declared local variables. We can use the same approach as described for globals to get details and documentation for the completion items.

4.2.2. Completion of Properties

Following the approach described in Section 3.3, we have implemented the *completion of properties* in our Graal-LSP prototype.

Detecting an Object Property Completion To distinguish a completion request for globals and locals from one for object properties, we can use the API extension already suggested in Section 4.1.3, which allows us to ask a Truffle language for its completion trigger characters. Because the *initialize* response contains the trigger characters of all installed languages, the client will automatically trigger completion requests for all these characters. Therefore we have to check every request, if the trigger character belongs to the current language. If not, we have to skip the request.

Including the completion trigger character in the request is an optional feature of the client. Furthermore, the character is only included if the completion was triggered automatically. To support code completion of properties also when triggered manu-

ally, we assume, that we can distinguish the two kinds of completion by comparing the character directly in front of the requested position with the registered trigger characters of the current language. If there is a match, we try an object property completion.

Finding the Desired Node As described in our approach, the source code will most likely not be parsable when a completion of properties is triggered. We assume that the last inserted text snippet is the completion trigger character, which caused the syntax error. Therefore, we simply store the last change for every `TextDocumentSurrogate` object to replace the last inserted text snippet with an empty string in the source code.

Having parsable source code, we can search for a node directly in front of the removed text snippet. We use the same approach as for finding the nearest node for a completion of locals and globals, but filter for nodes tagged with `ExpressionTag` only. If we find a `containingNode` whose source section ends directly in front of the removed text snippet, we take it to obtain its corresponding run-time object. In the following we refer to this node as `propertiesRequestedNode`.

Strategies to Obtain Properties The next step we have to do is getting the run-time object represented by the `propertiesRequestedNode`, to finally query that object for its properties. We obtain run-time information as described in Section 4.1.6, but we have implemented further optimizations for this kind of completion.

First of all we can optimize property completion for literal nodes in Truffle. For instrumentable nodes, we can use the method `getNodeObject()` to get a `TruffleObject`. We assume, that it supports reading a value for the key `"literal"`. If supported, we can use the read value to create completion items for its properties. If the value is a `TruffleObject`, we obtain its properties as described for globals and locals in Section 4.2.1. But the returned value might also be a primitive Java type, which needs to be boxed into a language-specific `TruffleObject`, so that we can query it for properties. Therefore, we added the method `boxPrimitive(String languageId, Object primitive)` to the instrumentation environment API.

If the literal-approach returns no results, we check for cached run-time information. To facilitate them, we use in-line evaluation, which requires that the Truffle language implementation supports in-line parsing. First of all, we traverse the AST, starting at the root node of our `propertiesRequestedNode`, to find a node in the neighborhood, which has a mapping to `CoverageData` in our cache. To call `parseInline(Source source, Node node, MaterializedFrame frame)`, we create a `Source` object by using the source section text of our `propertiesRequestedNode`. As second parameter, we use the `propertiesRequestedNode` itself. The `frame` is taken from the `CoverageData` object. If in-line parsing is successful, the returned result is an `ExecutableNode` instance. To execute that node, it needs a parent. Therefore, we temporarily insert it as child into the `ExecutionEventNode` which we was stored in the `CoverageData` object during the run-time collection run. The properties of the execution result are queried as described for literals in the previous paragraph.

If there are no cached information yet, we start executing source code and observing the execution until it reaches our `propertiesRequestedNode`. We interrupt the execution at this point as described in Section 4.1.6, to retrieve the evaluation result of the `propertiesRequestedNode`.

As last fallback, we try to parse and execute the source section text of the `propertiesRequestedNode` in the global scope of the language.

summary?

4.2.3. Signature Help

As described in our approach in Section 3.4, we have implemented signature help support similar to the completion of properties. We added the method `getSignatureHelpTriggerCharacters(String languageId)` to the instrumentation environment, to retrieve the needed trigger characters. In contrast to the completion of properties implementation, we assume that the source code is parsable after the trigger character is inserted, because many language clients support smart character insertion, so that for example the insertion of the widely used trigger character '(' is automatically amended to '()'. Furthermore, we do not search for a node having a source section in front of the trigger character, but for a node at the requested position, tagged with a `CallTag`.

If found, we execute the source code as for completion of properties, but we listen for slightly different events, because we want to obtain the callable object itself instead of the return value of the called callable. Therefore, we have implemented the `onInputValue(EventContext context, VirtualFrame frame, EventContext inputContext, int inputIndex, Object inputValue)` method of the listener, which provides intermediate results of expressions. In our case, we can intercept the callable object before it is called.

Having the callable, we send the added custom message `GetSignature` to retrieve a `TruffleObject` representing a list of signature parameters. As described in our approach, the language needs to provide a formatted string representation of the signature which we can use as label for the response. We assume therefore, that the retrieved `TruffleObject` has a `format()` method, which we can call by sending an `INVOKE` message.

To set the active parameter index of the response, we simply use the number of provided arguments. We assume, that the `CallTag`-annotated node can tell us that number. Therefore, we call `getNodeObject()`, which should return a `TruffleObject` supporting the reading of a value for the key `"numberOfArguments"`.

4.2.4. Hover Information

Our implementation of Graal-LSP supports hover information for variables and callables, if cached run-time information are available. We use the already described approach to find a node corresponding to the requested position. If found, we try two strategies to get a run-time object represented by that node.

First of all, we test if the text of the node's source section matches an identifier of one of the frame slots. We can access these slots by calling `frame.getFrameDescriptor().get` how to handle? If found, we use the value associated to that frame slot.

If not found, we in-line evaluate the text of the node's source section, as described for the completion of properties in Section 4.2.2.

If one of the strategies yields a result, we use again the same implementation to query the object for meta-information, signature and documentation as described for the completion of properties.

4.2.5. Document Symbols

To find symbol declarations in the current file, we added a new tag `DeclarationTag`. We use the instrumentation environment's `LoadSourceSectionListener` to search for all tagged nodes in the current `Source` object. Then, we assume that a call to `getNodeObject()` returns a `TruffleObject` supporting the look-up for the keys `"name"`, `"kind"`, and `"container"`. We can map these information together with the node's source section to the *documentSymbol* response items.

If a Truffle languages supports no `DeclarationTag`, we use a simple fallback. We use the instrumentation environment to search for all nodes tagged with the already existing `RootTag` in the current `Source` object. This way, we get at least names and source sections of declarations of callables, to fill the *documentSymbol* response items partially.

4.2.6. Workspace Symbols

To search for symbols workspace-wide, we use the same implementation as described for the *documentSymbol* request, but we omit the filter for the current `Source` object.

4.2.7. Document Highlight

We implemented *documentHighlight* support in Graal-LSPfor variables. To make variable nodes detectable, we added two new tags, `ReadVariableTag` and `WriteVariableTag`. We assume, that nodes tagged with one of them implement `getNodeObject()`, returning a `TruffleObject` which supports the look-up key `"name"`. We use the instrumentation environment's `LoadSourceSectionListener` to search for all tagged nodes in the current `Source` object.

If found, we use the instrumentation environment to find local scopes for these nodes. Then, we traverse the root node of each scope by calling `accept(NodeVisitor nodeVisitor)` and search for children tagged with either `ReadVariableTag` or `WriteVariableTag`. We take those nodes providing the same `"name"` as our initial node and build the response using their source sections.

4.2.8. Goto-definition

The *definition* request has been implemented for callables workspace-wide and for variables source-local. As described in the previous sections, we search for a node at the requested position, which is tagged with `CallTag`, `ReadVariableTag`, or `WriteVariableTag`. If found, we use different approaches for callables and variables.

For nodes of callables, we obtain the corresponding run-time object as described for the completion of properties in Section 4.2.2. Then, we use the method `findSourceLocation(LanguageInfo language, Object value)` of the instrumentation environment, to find the source section of the callable.

If the run-time approach yields no results, we use a static fallback search. As described in Section 4.2.6, we search for all symbols in the workspace and take the ones matching the identifier of our callable node. For non-unique identifiers, this might yield many results, which are not related to the desired definition location. But in our opinion, it is better to provide the users a list of potential matches instead of providing nothing.

ist das cha:evaluation?

For variables, we assume that the first write of a local variable is its definition location. This way, we can reuse the implementation described in Section 4.2.7, to find all other occurrences of a local variable and pick the first write access as definition location.

4.2.9. Finding All References

The *references* requests has been implemented for callables and local variables using a static search. As described in the previous sections, we search for a node at the requested position, which is tagged with `CallTag`, `ReadVariableTag`, or `WriteVariableTag`. If found, we use different approaches for callables and variables.

For nodes of callables, we reuse the static fallback search described in Section 4.2.8, to find all potential matches in the workspace.

For variables, we reuse the implementation described in Section 4.2.7, to find all occurrences of variable accesses in the current `Source`.

4.3. Implementation Guideline for Truffle Languages

Quality of LSP results depends on the support of the language implementation. Implementation guideline:

- eager source code parsing
- consistent source section hierarchy
- Implement existing API:
  - ExpressionTag + StatementTag
  - findTopScopes()
  - findLocalScopes()
  - findMetaObject()
  - findSourceLocation()

  - parseInline()
  - getNodeObject() for literals

- Implement proposed/new API:
  - getCompletionTriggerCharacters()
  - getSignatureHelpCharacters() (TODO)
  - boxPrimitive()
  - GetSignature custom message (+ format invoke + getNodeObject() "numberOfArguments" look-up)
  - GetDocumentation custom message
  - DefinitionTag + ReadVariableTag + WriteVariableTag

5. Experience Report

Eat your own dog food Hover: We used the static hover information ourselves during implementation.

GraalPython, SL, (GraalJS), FastR usage check

- matrix of Graal-LSPfeatures x Truffle languages
- list SLOC for adjustments of these langs
- matrix of Graal-LSPfeatures x Truffle languages without language adjustments? Should be only completion of globals/locals and static stuff possible -> or add a footnote in the first matrix, to mark that it is only working with adjustments
- include screenshots of VSCode for important LSP features, put rest in appendix?

Completion response time

Between IDE and Language Workbench Language Workbench focus/aim: Efficient language (DSL) development, performance is not main focus vs. Graal/Truffle aim: Efficient execution

Assumption: Language Workbenches use static information only, our Truffle-based solution uses dynamic type information

5.1. Limitations of Graal-LSP

- Server: Only TCP (connecting to already running server), no std-in/-out impl
- Truffle: Only first syntax error is reported. Lang parsers fail on first error
- Cancellation of requests not yet supported
- also, this in-line eval is interpretation only
- sandboxing limited to fie I/O + read-only FS atm
- limitations due to caching of last nested context - show example of dirty context, e.g. function renamed
- Using cached run-time infos: we have frame but to do in-line eval we call node.execute(frame) ourselves without having a valid execution stack there -> if the execution relies on the stack, it will not work
- type harvesting: we take a snapshot of the frame for the first entering of a source section -> different types for a frame slot are ignored
- the snapshot taken copies frame and descriptor, but no deep copy of objects is done. Therefore the reference to these objects will represent the last state of execution

5.2. Limitations of Our Approach

- LSP: Syntax highlighting defined at client-side
- LSP+VSCode: Need to define supported languages in extension up-front
- LSP: No incremental sending of (completion) results
- LSP: has file-based code focus (GraalSqueak integration really hard)
- Run-time type collection / type harvesting
  - framework needs to support instrumentation / tracing to find the desired run-time object
  - needs possibility to query the resulting host language object about its guest language semantics (e.g. a TruffleObject understanding a KEYS message)
  - we query the actual run-time object, which might be really specialized, whereas type harvesting builds generalizations on seen types and yields only the generalized type

6. Related Work

In this chapter, we describe approaches and solutions related to our work and compare them with our approach and implementation of Graal-LSP.

Our focus: out of the box IDE-like tooling support for performance-focused implementations of dynamic languages.

Outline

- Typical IDE-frameworks used by performance-focused language implementations
  - Eclipse
  - Visual Studio
  - IntelliJ
- Tools-first language implementation frameworks
  - Language workbenches
    - Spoofax and DynSem - generates a Truffle AST interpreter
  - Helvetia
  - Squimera
- Language-agnostic Approaches for IDE-like Features
  - Kythe
  - CTags
- Approaches Exploiting run-time information
  - Type harvesting
  - PyCharm - dynamic type inference for code completion
  - pyannotate - Insert annotations into your source code based on call arguments and return types observed at run-time

Outline old

- Motivation related (Tooling support out of the box for your language)
  - Helvetia - embedding dsls into Smalltalk, so that the tools provided by the environment can also be used for these languages
  - Squimera - interpreter composition to reuse existing Smalltalk tools
  - Language Workbenches - tools that support the efficient definition, reuse and composition of languages and their IDEs

  - ? Moldable Tools - development tools that are aware of the current development context and support inexpensive creation of domain-specific extensions.

- Graal-LSP related
  - Collecting run-time information
    * Type harvesting
    * Nemo Bachelor Project - sandboxed execution
    * PyCharm - dynamic type inference for code completion
    * pyannotate - Insert annotations into your source code based on call arguments and return types observed at run-time
  - Polyglot IDE support
    * DSLs: Language Workbenches, Helvetia, ...
    * Interpreter composition: Squimera, Ensō
    * -> but all of them bind you to one editor front-end
- LSP-related
  - Language-agnostic tagging of symbols
    * Kythe - A pluggable, (mostly) language-agnostic ecosystem for building tools that work with code. <- only "look-up tools"
    * CTags - tagging a language object in an index file

6.1. Reusing existing tools

Helvetia [11] -> Helvetia has no performance focus, Graal-LSPperformance -> Helvetia DSL focus, Graal-LSPgeneral purpose focus, but DSLs possible -> Helvetia allows composing DSLs in same source, Graal-LSPhas Polyglot support, but only string based -> DSLs can use Smalltalk environment tools, but none else -> Graal-LSPlet's you choose the lsp client -> Helvetia translates dsls to Smalltalk by adding hooks into the Smalltalk compiler chain, Graal-LSP, i.e. Truffle uses language-specific interpreters -> Helvetia DSLs: syntax highlighting, debugger, -> but code completion, goto-definition etc only at Smalltalk level

Ensō is also mentioned in Language Workbenches

Self

Smalltalk+Squeak

Squimera [10] [9]

6.2. Language Workbenches

[2] -> not performance focused -> typically only one or a very limited set of target IDEs supported (e.g. an Eclipse plugin) -> not limited to textual notation (also graphical, tabular etc.) -> Composability -> Testing / debugging -> Editing mode: free form + projectional -> Syntactic services (Highlighting, Outline, Folding, Syntactic completion, Diff, Auto formatting) -> Semantic services (Reference resolution, Semantic completion, Refactoring, Error marking, Quick fixes, Origin tracking, Live Translation)
- stefan marr:

- Xtext, part of the Eclipse project, is a framework for defining external DSLs in close cooperation with the Eclipse IDE. [6]
- [15] use Spoofax and DynSem [14] -> generated AST-interpreter for Truffle <- can one modify the generated interpreter? -> end-user IDE: bound to the Spoofax generated editor(-plugin)
- Ensō http://enso-lang.org -> Ensō is a self-describing workbench based entirely on interpreters implemented in an object-oriented language. Ensō avoids the generation of code and follows the hypothesizes, that interpreters are easier to develop and combine than compilers or transformational approaches. Because Ensō is implemented in itself, all available tools in the system can be reused to create applications. -> Smalltalk approach

6.3. Run-time type collection

- [5]

- Nemo Bachelor Project? -> sandboxed execution in Squeak
- https://github.com/dropbox/pyannotate -> Insert annotations into your source code based on call arguments and return types observed at runtime. - uses Python's sys.setprofile(profilefunc)
- https://blog.jetbrains.com/pycharm/2013/02/dynamic-runtime-type-inference-in-pycharm-2-7/

6.4. Language-agnostic tagging of symbols

Kythe

- A pluggable, (mostly) language-agnostic ecosystem for building tools that work with code.
- https://kythe.io/

-> LSP more specific, focus on IDE-like tooling missing in Truffle -> Kythe isn't really intended to be (or to replace) a UI-oriented tool like an editor or an IDE (LSP neither)
-> Since Kythe analysis is static, it requires code that builds. Completion, by design, is intended for code that is in-progress, and generally doesn't (yet) compile or run. One could certainly use data from a Kythe index to augment local completion results, but the Kythe data won't solve the local problem of generating symbols for completion.
-> but we explicitly want to support code completion

CTags

- tagging a language object in an index file
- tag-file supported by editors
- allows user to locate the tagged object via name (goto-definition, list symbols)
- for new languages you need to define a regex or parser to find language objects
- only static analysis, no scoped completion (everything is suggested)

SemanticDB

7. Conclusion and Future Work

Unique features

- Polyglot object property completion (e.g. suggest props of an JS object in python code)
- Goto-definition for functions which are first-class citizens

Will language implementers add needed interface impls to support most LSP features? We think yes, because you gain a lot with small changes and most of them are also needed for the Debugger (Expression Tags etc) or other tools

discuss this in Experience Report

7.1. Future Work

- Dynamic type collection
  - sandboxing -> copy-on-write FS
  - how to handle network I/O
  - smart caching -> store also frames which have different var types in a frame slot
- LSP features
  - Cancellation support for requests (In Truffle: ExecutionListener which checks each x statements if time limit exceeded -> throw Time-out-ThreadDeath-like exception)
  - Refactoring features
  - If semantic coloring is supported -> use that to compensate client-side syntax highlighting definition
- Truffle
  - how to enrich ASTs with parse-time infos without influencing run-time performance

Bibliography

[1] Muhammad Asaduzzaman. "Context-Sensitive Code Completion". PhD thesis. University of Saskatchewan, 2018.

[2] Sebastian Erdweg, Tijs Van Der Storm, Markus Völter, Meinte Boersma, Remi Bosman, William R Cook, Albert Gerritsen, Angelo Hulshout, Steven Kelly, Alex Loh, et al. "The State of the Art in Language Workbenches". In: *International Conference on Software Language Engineering*. Springer. Springer International Publishing, 2013, pp. 197–217. DOI: 10.1007/978-3-319-02654-1_11.

[3] Yoshihiko Futamura. "Partial evaluation of computation process–an approach to a compiler-compiler". In: *Higher-Order and Symbolic Computation* 12.4 (1999), pp. 381–391. DOI: 10.1023/A:1010095604496.

[4] Matthias Grimmer, Chris Seaton, Roland Schatz, Thomas Würthinger, and Hanspeter Mössenböck. "High-performance cross-language interoperability in a multi-language runtime". In: *ACM SIGPLAN Notices*. Vol. 51. 2. ACM. 2015, pp. 78–90. DOI: 10.1145/2936313.2816714.

[5] Michael Haupt, Michael Perscheid, and Robert Hirschfeld. "Type harvesting: a practical approach to obtaining typing information in dynamic programming languages". In: *Proceedings of the 2011 ACM Symposium on Applied Computing*. ACM. 2011, pp. 1282–1289. DOI: 10.1145/1982185.1982464.

[6] Christian Humer, Christian Wimmer, Christian Wirth, Andreas Wöß, and Thomas Würthinger. "A domain-specific language for building self-optimizing AST interpreters". In: *ACM SIGPLAN Notices* 50.3 (2015), pp. 123–132. DOI: 10.1145/2658761.2658776.

[7] Eva Maia, Nelma Moreira, and Rogério Reis. "A static type inference for python". In: *Proc. of DYLA* (2012).

[8] Microsoft. *Language Server Protocol*. https://microsoft.github.io/language-server-protocol/. Visited on 2018-09-24. 2017.

[9] Fabio Niephaus, Tim Felgentreff, Tobias Pape, and Robert Hirschfeld. "Squeak Makes a Good Python Debugger: Bringing Other Programming Languages Into Smalltalk's Tools". In: *Companion to the first International Conference on the Art, Science and Engineering of Programming*. ACM. 2017, p. 27. DOI: 10.1145/3079368.3079402.

[10] Fabio Niephaus, Tim Felgentreff, Tobias Pape, Robert Hirschfeld, and Marcel Taeumel. "Live Multi-language Development and Runtime Environments". In: *arXiv preprint arXiv:1803.10200* 2.3 (Mar. 2018). DOI: 10.22152/programming-journal.org/2018/2/8.

[11] Lukas Renggli, Tudor Gîrba, and Oscar Nierstrasz. "Embedding languages without breaking tools". In: *European Conference on Object-Oriented Programming*. Springer. 2010, pp. 380–404. DOI: 10.1007/978-3-642-14107-2_19.

[12] Michael L Van De Vanter. "Building debuggers and other tools: we can have it all". In: *Proceedings of the 10th Workshop on Implementation, Compilation, Optimization of Object-Oriented Languages, Programs and Systems*. ACM. 2015, p. 2. DOI: 10.1145/2843915.2843917.

[13] Michael Van De Vanter, Chris Seaton, Michael Haupt, Christian Humer, and Thomas Würthinger. "Fast, Flexible, Polyglot Instrumentation Support for Debuggers and other Tools". In: *arXiv preprint arXiv:1803.10201* (2018). DOI: 10.22152/programming-journal.org/2018/2/14.

[14] Vlad Vergu, Pierre Neron, and Eelco Visser. "DynSem: A DSL for Dynamic Semantics Specification". In: *26th International Conference on Rewriting Techniques and Applications (RTA 2015)*. Ed. by Maribel Fernández. Vol. 36. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2015, pp. 365–378. ISBN: 978-3-939897-85-9. DOI: 10.4230/LIPIcs.RTA.2015.365.

[15] Eelco Visser, Guido Wachsmuth, Andrew Tolmach, Pierre Neron, Vlad Vergu, Augusto Passalaqua, and Gabrieël Konat. "A Language Designer's Workbench: A One-Stop-Shop for Implementation and Verification of Language Designs". In: *Proceedings of the 2014 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software*. ACM. 2014, pp. 95–111. DOI: 10.1145/2661136.2661149.

[16] Christian Wimmer, Michael Haupt, Michael L Van De Vanter, Mick Jordan, Laurent Daynès, and Douglas Simon. "Maxine: An approachable virtual machine for, and in, java". In: *ACM Transactions on Architecture and Code Optimization (TACO)* 9.4 (2013), p. 30. DOI: 10.1145/2400682.2400689.

[17] Christian Wimmer and Thomas Würthinger. "Truffle: a self-optimizing runtime system". In: *Proceedings of the 3rd annual conference on Systems, programming, and applications: software for humanity*. ACM. 2012, pp. 13–14. DOI: 10.1145/2384716.2384723.

[18] Thomas Würthinger, Christian Wimmer, Christian Humer, Andreas Wöß, Lukas Stadler, Chris Seaton, Gilles Duboscq, Doug Simon, and Matthias Grimmer. "Practical Partial Evaluation for High-performance Dynamic Language Runtimes". In: *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation - PLDI 2017*. Vol. 52. 6. ACM. 2017, pp. 662–676. DOI: 10.1145/3062341.3062381.

[19] Thomas Würthinger, Christian Wimmer, Andreas Wöß, Lukas Stadler, Gilles Duboscq, Christian Humer, Gregor Richards, Doug Simon, and Mario Wolczko. "One VM to rule them all". In: *Proceedings of the 2013 ACM international symposium on New ideas, new paradigms, and reflections on programming & software*. ACM. 2013, pp. 187–204. DOI: 10.1145/2509578.2509581.

[20] Thomas Würthinger, Andreas Wöß, Lukas Stadler, Gilles Duboscq, Doug Simon, and Christian Wimmer. "Self-optimizing AST interpreters". In: *ACM SIGPLAN Notices*. Vol. 48. 2. ACM. 2012, pp. 73–82. DOI: 10.1145/2384577.2384587.

Appendix A.

First Unimportant stuff.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbständig verfasst sowie keine anderen Quellen und Hilfsmittel als die angegebenen benutzt habe.

Berlin, den 26. Dezember 2018

Daniel Stolpe

Todo list

- footnote? . . . 5
- footnote? . . . 5
- ref . . . 6
- Is this important? . . . 7
- consume in a "Virtual Execution Environment" section? . . . 8
- ref Frame: "Advanced Compiler Design and Implementation" book p.114 . . . 9
- ref to Fowler, Spoofax, Xtext etc - footnote vs ref? . . . 9
- ref . . . 9
- cite or footnote? . . . 10
- ref . . . 15
- far from perfect, difficult to built in a language-agnostic way, ... 15
- explain in background? . . . 16
- add remark: approach only possible for languages which support "expression statements"? <- how can we name that? For example in Java this would not work. . . . 18
- is not fitting. Defer to impl? . . . 18
- hier noch mehr erklären - Beispiel geben mit frame usw oder dann in Impl erläutern? . . . 19
- Sum up / Überleitung . . . 19
- Zusammenfassung, was wiederverwendet werden kann: run-to-section + static search for tagged IR-elements + IR-element lexical source section . . . 19
- Punkt am Ende? . . . 20
- name a version? . . . 23
- context? . . . 23
- figure showing a typical requests flowing through the different Threads . . . 24
- das muss irgendwo noch hin -> optimization: reuse Contexts, new Context erst wenn coverage analysis is triggered. Aber passt hier auch noch nicht so richtig. wohin? . . . 24
- execution of source code -or- evaluation of source code? . . . 26
- @Tim/Fabio how to handle that? . . . 27
- name limitation here, that captured frame snapshot is no deep copy? . . . 28
- class diagram? . . . 28
- label counter wrrong? . . . 28
- IntelliJ code completion uses a dummy reference, if there is none at the caret - should we name that here, that we cannot do that? . . . 29
- summary? . . . 32
- how to handle? . . . 33
- ist das cha:evaluation? . . . 34

discuss this in Experience Report 43